

Problems

13-1 *Persistent dynamic sets*

During the course of an algorithm, you sometimes find that you need to maintain past versions of a dynamic set as it is updated. We call such a set *persistent*. One way to implement a persistent set is to copy the entire set whenever it is modified, but this approach can slow down a program and also consume a lot of space. Sometimes, you can do much better.

Consider a persistent set S with the operations INSERT, DELETE, and SEARCH, which you implement using binary search trees as shown in Figure 13.8(a). Maintain a separate root for every version of the set. In order to insert the key 5 into the set, create a new node with key 5. This node becomes the left child of a new node with key 7, since you cannot modify the existing node with key 7. Similarly, the new node with key 7 becomes the left child of a new node with key 8 whose right child is the existing node with key 10. The new node with key 8 becomes, in turn, the right child of a new root r' with key 4 whose left child is the existing node with key 3. Thus, you copy only part of the tree and share some of the nodes with the original tree, as shown in Figure 13.8(b).

Assume that each tree node has the attributes *key*, *left*, and *right* but no parent. (See also Exercise 13.3-6 on page 346.)

- a.** For a persistent binary search tree (not a red-black tree, just a binary search tree), identify the nodes that need to change to insert or delete a node.

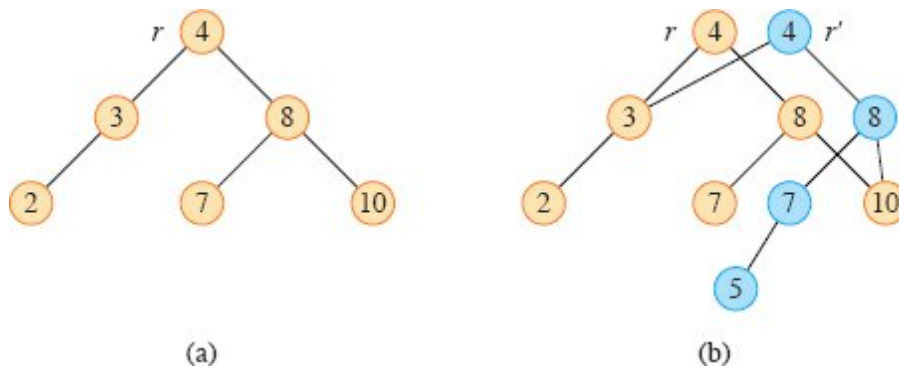


Figure 13.8 (a) A binary search tree with keys 2, 3, 4, 7, 8, 10. (b) The persistent binary search tree that results from the insertion of key 5. The most recent version of the set consists of the nodes reachable from the root r' , and the previous version consists of the nodes reachable from r . Blue nodes are added when key 5 is inserted.

- b.** Write a procedure **PERSISTENT-TREE-INSERT**(T, z) that, given a persistent binary search tree T and a node z to insert, returns a new persistent tree T' that is the result of inserting z into T . Assume that you have a procedure **COPY-NODE**(x) that makes a copy of node x , including all of its attributes.
- c.** If the height of the persistent binary search tree T is h , what are the time and space requirements of your implementation of **PERSISTENT-TREE-INSERT**? (The space requirement is proportional to the number of nodes that are copied.)
- d.** Suppose that you include the parent attribute in each node. In this case, the **PERSISTENT-TREE-INSERT** procedure needs to perform additional copying. Prove that **PERSISTENT-TREE-INSERT** then requires $\Omega(n)$ time and space, where n is the number of nodes in the tree.
- e.** Show how to use red-black trees to guarantee that the worst-case running time and space are $O(\lg n)$ per insertion or deletion. You may assume that all keys are distinct.

13-2 Join operation on red-black trees

The *join* operation takes two dynamic sets S_1 and S_2 and an element x such that for any $x_1 \in S_1$ and $x_2 \in S_2$, we have $x_1.key \leq x.key \leq x_2.key$. It returns a set $S = S_1 \cup \{x\} \cup S_2$. In this problem, we investigate how to implement the join operation on red-black trees.

- a.** Suppose that you store the black-height of a red-black tree T as the new attribute $T.bh$. Argue that RB-INSERT and RB-DELETE can maintain the bh attribute without requiring extra storage in the nodes of the tree and without increasing the asymptotic running times. Show how to determine the black-height of each node visited while descending through T , using $O(1)$ time per node visited.

Let T_1 and T_2 be red-black trees and x be a key value such that for any nodes x_1 in T_1 and x_2 in T_2 , we have $x_1.key \leq x.key \leq x_2.key$. You will show how to implement the operation $\text{RB-JOIN}(T_1, x, T_2)$, which destroys T_1 and T_2 and returns a red-black tree $T = T_1 \cup \{x\} \cup T_2$. Let n be the total number of nodes in T_1 and T_2 .

- b.** Assume that $T_1.bh \geq T_2.bh$. Describe an $O(\lg n)$ -time algorithm that finds a black node y in T_1 with the largest key from among those nodes whose black-height is $T_2.bh$.
- c.** Let T_y be the subtree rooted at y . Describe how $T_y \cup \{x\} \cup T_2$ can replace T_y in $O(1)$ time without destroying the binary-search-tree property.
- d.** What color should you make x so that red-black properties 1, 3, and 5 are maintained? Describe how to enforce properties 2 and 4 in $O(\lg n)$ time.
- e.** Argue that no generality is lost by making the assumption in part (b). Describe the symmetric situation that arises when $T_1.bh \leq T_2.bh$.
- f.** Argue that the running time of RB-JOIN is $O(\lg n)$.

13-3 AVL trees

An *AVL tree* is a binary search tree that is *height balanced*: for each node x , the heights of the left and right subtrees of x differ by at most 1. To implement an AVL tree, maintain an extra attribute h in each node such that $x.h$ is the height of node x . As for any other binary search tree T , assume that $T.root$ points to the root node.

- a.** Prove that an AVL tree with n nodes has height $O(\lg n)$. (*Hint*: Prove that an AVL tree of height h has at least F_h nodes, where F_h is the h th Fibonacci number.)
- b.** To insert into an AVL tree, first place a node into the appropriate place in binary search tree order. Afterward, the tree might no longer be height balanced. Specifically, the heights of the left and right children of some node might differ by 2. Describe a procedure $BALANCE(x)$, which takes a subtree rooted at x whose left and right children are height balanced and have heights that differ by at most 2, so that $|x.right.h - x.left.h| \leq 2$, and alters the subtree rooted at x to be height balanced. The procedure should return a pointer to the node that is the root of the subtree after alterations occur. (*Hint*: Use rotations.)
- c.** Using part (b), describe a recursive procedure $AVL-INSERT(T, z)$ that takes an AVL tree T and a newly created node z (whose key has already been filled in), and adds z into T , maintaining the property that T is an AVL tree. As in $TREE-INSERT$ from Section 12.3, assume that $z.key$ has already been filled in and that $z.left = \text{NIL}$ and $z.right = \text{NIL}$. Assume as well that $z.h = 0$.
- d.** Show that $AVL-INSERT$, run on an n -node AVL tree, takes $O(\lg n)$ time and performs $O(\lg n)$ rotations.

Chapter notes

The idea of balancing a search tree is due to Adel'son-Vel'skiĭ and Landis [2], who introduced a class of balanced search trees called “AVL trees” in 1962, described in Problem 13-3. Another class of search trees, called “2-3 trees,” was introduced by J. E. Hopcroft (unpublished) in

1970. A 2-3 tree maintains balance by manipulating the degrees of nodes in the tree, where each node has either two or three children. Chapter 18 covers a generalization of 2-3 trees introduced by Bayer and McCreight [39], called “B-trees.”

Red-black trees were invented by Bayer [38] under the name “symmetric binary B-trees.” Guibas and Sedgewick [202] studied their properties at length and introduced the red/black color convention. Andersson [16] gives a simpler-to-code variant of red-black trees. Weiss [451] calls this variant AA-trees. An AA-tree is similar to a red-black tree except that left children can never be red.

Sedgewick and Wayne [402] present red-black trees as a modified version of 2-3 trees in which a node with three children is split into two nodes with two children each. One of these nodes becomes the left child of the other, and only left children can be red. They call this structure a “left-leaning red-black binary search tree.” Although the code for left-leaning red-black binary search trees is more concise than the red-black tree pseudocode in this chapter, operations on left-leaning red-black binary search trees do not limit the number of rotations per operation to a constant. This distinction will matter in Chapter 17.

Treaps, a hybrid of binary search trees and heaps, were proposed by Seidel and Aragon [404]. They are the default implementation of a dictionary in LEDA [324], which is a well-implemented collection of data structures and algorithms.

There are many other variations on balanced binary trees, including weight-balanced trees [344], k -neighbor trees [318], and scapegoat trees [174]. Perhaps the most intriguing are the “splay trees” introduced by Sleator and Tarjan [418], which are “self-adjusting.” (See Tarjan [429] for a good description of splay trees.) Splay trees maintain balance without any explicit balance condition such as color. Instead, “splay operations” (which involve rotations) are performed within the tree every time an access is made. The amortized cost (see Chapter 16) of each operation on an n -node tree is $O(\lg n)$. Splay trees have been conjectured to perform within a constant factor of the best offline rotation-based tree. The best known competitive ratio (see Chapter 27) for a rotation-based tree is the Tango Tree of Demaine et al. [109].

Skip lists [369] provide an alternative to balanced binary trees. A skip list is a linked list that is augmented with a number of additional pointers. Each dictionary operation runs in $O(\lg n)$ expected time on a skip list of n items.

¹ Although we try to avoid gendered language in this book, the English language lacks a gender-neutral word for a parent's sibling.

² If property 5 holds, we can assume that paths from the roots of γ , δ , ϵ , and ζ down to leaves contain one more black than do paths from the roots of α and β down to leaves.



Part IV Advanced Design and Analysis Techniques

Introduction

This part covers three important techniques used in designing and analyzing efficient algorithms: dynamic programming (Chapter 14), greedy algorithms (Chapter 15), and amortized analysis (Chapter 16). Earlier parts have presented other widely applicable techniques, such as divide-and-conquer, randomization, and how to solve recurrences. The techniques in this part are somewhat more sophisticated, but you will be able to use them solve many computational problems. The themes introduced in this part will recur later in this book.

Dynamic programming typically applies to optimization problems in which you make a set of choices in order to arrive at an optimal solution, each choice generates subproblems of the same form as the original problem, and the same subproblems arise repeatedly. The key strategy is to store the solution to each such subproblem rather than recompute it. Chapter 14 shows how this simple idea can sometimes transform exponential-time algorithms into polynomial-time algorithms.

Like dynamic-programming algorithms, greedy algorithms typically apply to optimization problems in which you make a set of choices in order to arrive at an optimal solution. The idea of a greedy algorithm is to make each choice in a locally optimal manner, resulting in a faster algorithm than you get with dynamic programming. Chapter 15 will help you determine when the greedy approach works.

The technique of amortized analysis applies to certain algorithms that perform a sequence of similar operations. Instead of bounding the

cost of the sequence of operations by bounding the actual cost of each operation separately, an amortized analysis provides a worst-case bound on the actual cost of the entire sequence. One advantage of this approach is that although some operations might be expensive, many others might be cheap. You can use amortized analysis when designing algorithms, since the design of an algorithm and the analysis of its running time are often closely intertwined. Chapter 16 introduces three ways to perform an amortized analysis of an algorithm.